

Trabalho P1 – Biblioteca de Grafos em C++

Teoria dos Grafos – COS 242

Carlos Bruno Arthur Cury

<https://github.com/Ludwigo0/TrabalhoGrafos>

2026

A ideia do projeto

- O objetivo não era apenas criar um executável: era construir uma **biblioteca que pudesse ser reutilizada** em outros programas.
- **libgrafo** guarda as representações e os algoritmos. Ela não conhece a linha de comando nem os arquivos de saída.
- **grafo** é o programa-cliente: recebe as opções, chama a biblioteca, mede os tempos e grava os resultados.
- Essa separação facilita tanto a reutilização quanto os testes.

Como as classes se conectam

- **Graph** é a interface comum: define o que qualquer representação precisa oferecer.
- **CSRGraph** implementa a lista de adjacência. **BitMatrixGraph** implementa a matriz em bits.
- BFS, DFS, componentes e distâncias recebem apenas um `const Graph&`. Eles não precisam conhecer a estrutura interna.
- Para repetir o mesmo experimento, basta trocar `--rep lista` por `--rep matriz`.
- Na chamada, o programa sempre enxerga um **Graph**. Internamente, as funções `*Auto` identificam se o objeto é um CSR ou uma matriz de bits e selecionam o tratamento adequado.
- A matriz precisa de uma otimização específica, porque a interface genérica acessa vizinhos individualmente; sem cuidado, a mesma linha poderia ser percorrida repetidamente.
- A interface e o resultado não mudam; a especialização fica escondida na biblioteca.

A interface Graph em código

```
class Graph {  
public:  
    virtual ~Graph() = default;  
    virtual int numVertices() const = 0;  
    virtual long long numEdges() const = 0;  
    virtual int degree(int v) const = 0;  
    virtual int neighbor(int v, int k) const = 0;  
};
```

- A classe base não guarda nenhum vetor: ela define apenas o contrato.
- O algoritmo pergunta “qual é o grau?” e “qual é o vizinho k ?”, sem perguntar “onde isso está armazenado?”.
- As duas representações podem compartilhar o mesmo código de alto nível, enquanto a travessia adequada é escolhida internamente.
- Essa escolha acontece em tempo de execução, antes da busca começar; não precisamos criar um segundo programa ou duplicar o cliente.

Por que escolhemos CSR?

- Uma lista de adjacência guarda apenas as arestas que realmente existem.
- A alternativa simples, `vector<vector<int>>`, cria um vetor para cada vértice. Cada vetor custa cerca de 24 bytes mesmo quando está vazio.
- No maior grafo, isso representa aproximadamente **115 MB apenas para administrar os vetores**, antes das arestas.
- CSR usa dois vetores contíguos e chega a aproximadamente **373 MB no grafo 6**, com tamanho exato e boa localidade de cache.
- Como o grafo é montado uma vez e depois apenas consultado, a estrutura estática atende bem ao nosso caso de uso.

CSR: como os dois vetores funcionam

```
class CSRGraph : public Graph {
    vector<int> offsets_; // tamanho  $n + 1$ 
    vector<int> edges_;  // tamanho  $2m$ 

    int degree(int v) const override {
        return offsets_[v + 1] - offsets_[v];
    }

    int neighbor(int v, int k) const override {
        return edges_[offsets_[v] + k];
    }
};
```

- `offsets[v]` marca o começo da lista de `v`; `offsets[v+1]` marca o fim.

Exemplo pequeno

```
offsets = {0,2,3,5}   edges = {1,2 | 0 | 0,1}
v=0: edges[0..2) = {1,2}   v=1: edges[2..3) = {0}
v=2: edges[3..5) = {0,1}
```

- A carga ocorre em duas passadas: conta graus, calcula os offsets e depois preenche edges; assim não mantemos 46 milhões de arestas temporárias.

Matriz em bits: a ideia principal

- A matriz reserva uma posição para cada par de vértices, mesmo quando não existe aresta. Por isso seu custo depende de n^2 .
- Em vez de usar um byte por posição, guardamos apenas um bit: **0 = não existe aresta; 1 = existe aresta.**
- No grafo 2, isso reduz a estimativa de 2,3 GB para **297 MB**.
- O ganho de memória vem com um acesso um pouco mais elaborado; esse detalhe fica encapsulado na biblioteca.
- A partir do grafo 3, mesmo essa matriz compacta exigiria memória demais, por isso o programa recusa a alocação e registra OOM teórico.

Matriz em bits: como o acesso muda

Matriz clássica

Cada posição usa um byte. Para consultar (u, v) , calculamos a posição da célula e lemos diretamente o valor 0 ou 1.

```
matriz[u * n + v]
```

Matriz em bits

Cada palavra guarda 64 posições. Primeiro localizamos o bloco e depois o bit dentro dele.

```
size_t p = (size_t)u * n_ + v;  
bool existe = (data_[p / 64] >> (p % 64)) & 1  
ULL;
```

- A versão em bits economiza oito vezes mais memória, mas precisa de divisão, deslocamento e máscara para consultar uma posição.
- Por isso BitMatrixGraph possui um tratamento interno próprio para percorrer linhas; essa complexidade não aparece para quem usa Graph.

Quando os dados ficaram grandes

- O grafo 6 possui aproximadamente 93 milhões de inteiros em 722 MB de texto. Usamos um leitor próprio por blocos com `fread`.
- BFS e DFS são iterativas: uma DFS recursiva poderia estourar a pilha.
- O cronômetro envolve apenas o algoritmo, sem leitura ou escrita. A memória de pico é registrada pelo próprio processo.
- O diâmetro exato faria 4,8 milhões de BFSs no grafo 6, mais de 30 dias na taxa medida. Por isso também implementamos o *double-sweep*, com apenas duas BFSs.

Double-sweep: estimativa do diâmetro

Primeira BFS: origem qualquer $s \rightarrow$ mais distante a

Segunda BFS: origem $a \rightarrow$ mais distante b

Resultado: $d(a, b) = \text{nível de } b \text{ na segunda BFS}$

Memória: o que coube na prática?

Grafo	Lista CSR	Matriz em bits
1 (10 mil v.)	6 MB	16,8 MB
2 (50 mil v.)	15 MB	302 MB
3-4 (375 mil v.)	15 / 72 MB	OOM (~17,6 GB)
5-6 (4,8 milhões v.)	161 / 415 MB	OOM (~2,93 TB)

- OOM significa que a matriz exigiria mais memória do que a máquina consegue fornecer; não é uma falha silenciosa do programa.
- A comparação completa entre representações é fisicamente possível apenas nos grafos 1 e 2.

Tempo médio por busca (ms)

Grafo	BFS lista	BFS matriz	DFS lista	DFS matriz
1	0,8	4,9	1,0	6,3
2	2,6	28,5 ($\sim 11\times$)	4,4	32,1
3	10,5	—	18,5	—
4	106	—	80	—
5	863	—	1366	—
6	606	—	905	—

- Média de 100 buscas, sem contar I/O.
- Lista: $O(n + m)$; matriz em bits: $O(n^2)$ por busca.
- No grafo 2, a matriz cabe, mas na BFS já leva aproximadamente 11 vezes mais (na DFS, cerca de 7 vezes).

O que os dados revelaram

- Os grafos não são todos conexos: o grafo 2 tem 10 componentes; os grafos 3 e 4 têm 2; os grafos 5 e 6 têm 5. Por isso algumas distâncias retornam -1 : não existe caminho entre os vértices.
- Há arestas repetidas em ordem invertida (52 no grafo 1 e 1019 no grafo 2). Isso produz pequenas diferenças no grau médio (0,047% no grafo 1 e 0,078% no grafo 2), mas não altera as conclusões dos algoritmos.
- Nas representações `CSRGraph` e `BitMatrixGraph`, os níveis da BFS coincidem porque representam distâncias mínimas. Os pais podem mudar quando há mais de um predecessor possível; a ordem de visita define qual deles é escolhido.

- A matriz é interessante nos dois primeiros grafos, mas deixa de ser viável a partir do terceiro. A CSR é a representação que permite tratar todos os dados.
- A interface comum permite reutilizar os algoritmos. As “vias rápidas” são implementações internas específicas da matriz que percorrem seus bits sem repetir a mesma varredura; o programa e os resultados continuam iguais.
- Assim, a matriz não é penalizada por uma implementação ingênua da interface: a comparação mede as estruturas, não um desperdício evitável.
- O script refaz os experimentos, a suíte verifica exemplos e casos de erro, e o código completo está no repositório.

<https://github.com/Ludwigo0/TrabalhoGrafos>

Obrigado!

Carlos Bruno – Arthur Cury